

Performanceonderzoek naar hostingarchitecturen voor Studententuin

Definitieve versie van het eindverslag

Project: Studententuin

Module: Performance onderzoek

Datum: 1 april 2026

Versie: definitief

Samenvatting

Studententuin is een concept voor een betaalbaar hostingplatform waarop gebruikers eigen applicaties kunnen beheren. Binnen zo'n platform is de keuze voor de onderliggende hostingarchitectuur belangrijk, omdat deze invloed heeft op prestaties, resourcegebruik, isolatie en schaalbaarheid. In dit onderzoek is daarom bekeken welke van drie hostingarchitecturen het meest geschikt is voor Studententuin: traditionele containers met runc, sandboxed containers met gVisor en microVM-gebaseerde containers met Kata Containers.

Het literatuuronderzoek laat zien dat deze drie varianten een duidelijk verschillend technisch uitgangspunt hebben. Traditionele containers delen de kernel van de host en bieden daardoor doorgaans de laagste overhead. gVisor plaatst een extra sandboxlaag tussen container en hostkernel en verhoogt zo de isolatie, maar introduceert ook extra overhead. Kata Containers gebruikt lightweight virtual machines en biedt daardoor sterkere isolatie, maar verwacht eveneens extra kosten in opstarttijd en resourceverbruik.

De praktijkmetingen laten een gemengd beeld zien. Bij cold starts is runc het snelst met een gemiddelde opstarttijd van 1006.4 ms, gevolgd door gVisor met 1272.4 ms en Kata Containers met 1959.7 ms. Onder load laat juist Kata Containers de hoogste gemiddelde doorvoer zien met 157.8 requests per seconde, gevolgd door gVisor met 109.3 requests per seconde en runc met 38.8 requests per seconde. Alle loadtestruns eindigden zonder mislukte requests.

De interpretatie van de resultaten vraagt wel voorzichtigheid. De resourcemetingen zijn verzameld via docker stats, waardoor de gemeten geheugencijfers voor Kata Containers waarschijnlijk niet de volledige host-overhead van de microVM meenemen. Daardoor zijn vooral de cold-startresultaten en de loadtestresultaten het meest bruikbaar voor de vergelijking. Op basis van de combinatie van literatuur en metingen lijkt runc de meest logische keuze voor een eerste versie van Studententuin wanneer snelle opstart en eenvoud centraal staan. Kata Containers laat echter zien dat microVM-gebaseerde isolatie in deze proof of concept niet automatisch tot slechtere loadprestaties hoeft te leiden.

1. Introductie

Studententuin is een projectidee voor een hostingplatform waarop gebruikers eigen applicaties kunnen aanmaken, beheren en monitoren. Binnen het concept staat centraal dat het platform betaalbaar en beheersbaar moet blijven, terwijl gebruikers wel voldoende controle krijgen over hun eigen applicaties. Daardoor speelt de hostinglaag een centrale rol: daar moeten applicaties gestart, gestopt, gemonitord en aangepast kunnen worden.

Voor Studententuin is de keuze van de hostingarchitectuur daarom geen puur technische detailkeuze, maar een ontwerpbeslissing met directe invloed op kosten, prestaties, veiligheid en uitbreidbaarheid. Een goedkope architectuur met zeer lage overhead is aantrekkelijk voor een studentproject of MVP, maar kan minder sterke isolatie bieden. Een zwaardere architectuur kan veiliger zijn, maar is mogelijk minder efficiënt en daardoor duurder in gebruik.

Het doel van dit onderzoek is om drie relevante hostingarchitecturen te vergelijken op basis van prestatie-indicatoren die voor Studententuin praktisch relevant zijn. De vergelijking richt zich op traditionele containers met runc, sandboxed containers met gVisor en microVM-gebaseerde containers met Kata Containers. Deze drie varianten zijn gekozen omdat ze inhoudelijk duidelijk van elkaar verschillen in isolatiemodel en verwachte overhead.

De centrale gedachte achter het onderzoek is dat Studententuin niet de technisch meest geavanceerde oplossing nodig heeft, maar de oplossing die het best past bij de eisen van een klein, beheersbaar en betaalbaar hostingplatform. Daarom wordt niet alleen gekeken naar theorie, maar ook naar meetbare indicatoren zoals opstarttijd, geheugengebruik en prestaties onder belasting.

2. Literatuuronderzoek

2.1 Containers en traditionele runtime met runc

Docker beschrijft een container als een geïsoleerd proces met alle benodigde bestanden om te draaien. Wanneer meerdere containers op dezelfde host draaien, delen zij dezelfde kernel. Dat maakt containers efficiënt in resourcegebruik en opstarttijd, omdat er geen volledige gastbesturingssystemen gestart hoeven te worden (Docker, 2026a).

Docker Engine gebruikt standaard runc als container runtime. Docker documenteert daarnaast dat alternatieve runtimes via de daemonconfiguratie of via containerd shims gebruikt kunnen worden. Vanuit onderzoeksperspectief maakt dit runc een logisch referentiepunt: het is de standaardcontainer-runtime en vormt daarmee de baseline waartegen andere varianten kunnen worden vergeleken (Docker, 2026b).

Voor Studententuin heeft een runc-gebaseerde aanpak duidelijke voordelen. Containers met gedeelde hostkernel hebben doorgaans een lage overhead, kunnen snel starten en sluiten goed aan bij een platform waar veel kleine applicaties efficiënt naast elkaar moeten kunnen draaien. Het belangrijkste nadeel is dat de isolatie beperkter is dan bij oplossingen die een extra sandboxlaag of virtualisatielaag toevoegen.

2.2 gVisor als sandboxed containerruntime

gVisor positioneert zichzelf als een extra isolatielaag voor containers. De runtime runcs integreert met Docker en Kubernetes, zodat bestaande containerworkflow grotendeels behouden kan blijven. In plaats van systeemaanroepen direct aan de hostkernel door te

geven, plaatst gVisor daar een eigen userspace-kernel tussen. Daardoor wordt het aanvalsoppervlak richting de host verkleind (gVisor, 2026a; gVisor, 2026b).

Die extra beveiligingslaag heeft echter consequenties voor performance. In de eigen performance guide geeft gVisor aan dat de extra afhandeling van systeemaanroepen en I/O overhead kan veroorzaken, afhankelijk van de workload. De literatuur suggereert daarmee dat gVisor vooral interessant is in situaties waar sterkere isolatie gewenst is, maar waar volledige virtualisatie mogelijk nog te zwaar is (gVisor, 2026c).

Voor Studententuin is gVisor relevant omdat het een middenpositie vertegenwoordigt. Het behoudt een containergerichte workflow, maar probeert tegelijk een deel van de veiligheidsvoordelen van sterkere isolatie te bieden. De verwachting op basis van de literatuur is daarom dat gVisor minder efficiënt zal zijn dan runc, maar waarschijnlijk lichter blijft dan microVM-gebaseerde oplossingen.

2.3 Kata Containers en microVM-gebaseerde isolatie

Kata Containers wordt in de officiële documentatie omschreven als een secure container runtime met lightweight virtual machines. Het doel is om containers te laten werken zoals gewone containers in de toolchain, maar met sterkere workload-isolatie door hardwarevirtualisatie als extra verdedigingslaag te gebruiken. De officiële projectsite vat dit samen als lichte VMs die aanvoelen als containers (Kata Containers, 2026).

Het microVM-denken sluit aan bij Firecracker, een virtualisatiemodel dat is ontwikkeld voor lichte, snel startende virtuele machines voor grootschalige serverless- en containerachtige workloads. In de Firecracker-paper wordt beschreven dat microVMs bedoeld zijn om een betere balans te vinden tussen isolatie en efficiëntie dan traditionele virtuele machines (Agache et al., 2020).

Voor Studententuin is Kata Containers vooral interessant wanneer veiligheid en tenant-isolatie zwaarder wegen dan pure efficiëntie. Tegelijk ligt het op basis van de literatuur voor de hand dat deze architectuur meer geheugen gebruikt en hogere opstartkosten heeft dan standaardcontainers. Dat maakt Kata Containers relevant als derde variant in de vergelijking: niet als directe vervanger van runc, maar als architectuur aan het andere uiteinde van het spectrum.

2.4 Relevantie voor Studententuin

Voor Studententuin is de kernvraag niet welke technologie in het algemeen het best is, maar welke architectuur het best past bij een klein platform met beperkte middelen en meerdere gebruikersapplicaties. In die context zijn snelle starts, beheersbaarheid, duidelijke isolatie en voorspelbaar resourcegebruik belangrijker dan alleen de hoogste theoretische prestatie.

De literatuur wijst op een duidelijke trade-off. runc maximaliseert efficiëntie, gVisor voegt een extra sandboxlaag toe met verwachte overhead, en Kata Containers verschuift verder richting sterke isolatie met nog meer overhead. Deze tegenstelling maakt de drie

gekozen architecturen geschikt voor een toegepast performanceonderzoek met een proof of concept.

3. Onderzoeksvraag en deelvragen

Op basis van het literatuuronderzoek en de projectcontext van Studententuin is de volgende hoofdvraag geformuleerd:

Welke van drie hostingarchitecturen - traditionele containers met runc, sandboxed containers met gVisor, of microVM-gebaseerde containers met Kata Containers - is het meest geschikt voor Studententuin op basis van opstarttijd, resourcegebruik en prestaties onder belasting?

Deze hoofdvraag is opgesplitst in de volgende deelvragen:

Welke technische verschillen tussen runc, gVisor en Kata Containers zijn volgens de literatuur het meest relevant voor Studententuin?

Hoe verschillen de drie architecturen in cold-starttijd wanneer dezelfde applicatie wordt gestart?

Hoe verschillen de drie architecturen in idle geheugengebruik en idle CPU-gebruik wanneer dezelfde applicatie actief maar onbelast draait?

Hoe verschillen de drie architecturen in doorvoer en gemiddelde responstijd onder dezelfde loadtest?

Welke architectuur biedt, op basis van de combinatie van literatuur en metingen, de beste balans tussen efficiëntie en isolatie voor Studententuin?

De deelvragen sluiten direct aan op de hoofdvraag. Eerst worden de relevante technische verschillen uit de literatuur vastgesteld. Daarna worden precies die verschillen meetbaar gemaakt in een prototype. Op die manier vormen de deelvragen samen een logische route naar de beantwoording van de hoofdvraag.

4. Onderzoeksopzet en methodes

Dit onderzoek heeft een toegepast en vergelijkend karakter. Er wordt één identieke testapplicatie in drie verschillende hostingarchitecturen uitgevoerd. Door alleen de runtime te variëren en de applicatie, hostmachine en testcondities gelijk te houden, kan de invloed van de gekozen architectuur zo zuiver mogelijk worden vergeleken.

Het prototype bestaat uit een kleine webapplicatie met een health-endpoint en een endpoint dat lichte CPU-belasting simuleert. Deze applicatie wordt in alle drie de architecturen op exact dezelfde manier aangeboden. De drie onderzochte varianten zijn runc als standaardruntime, gVisor via runsc en Kata Containers via de daarvoor bedoelde runtime-integratie.

De dataverzameling richt zich op vier indicatoren. Ten eerste wordt de cold-starttijd gemeten als de tijd tussen het startcommando en de eerste succesvolle respons op het health-endpoint. Ten tweede wordt het idle geheugengebruik gemeten wanneer de container draait zonder actieve load. Ten derde wordt het idle CPU-gebruik gemeten in dezelfde situatie. Ten vierde wordt onder een gelijke loadtest gemeten hoeveel requests per second verwerkt kunnen worden en wat de gemiddelde responstijd is.

Om de betrouwbaarheid te vergroten worden metingen per architectuur meerdere keren herhaald. In de definitieve onderzoeksuitvoering zullen gemiddelden en standaarddeviaties worden berekend. De resultaten worden vervolgens grafisch weergegeven, zodat trends en verschillen direct zichtbaar zijn.

In deze definitieve versie zijn de voorbeeldresultaten vervangen door daadwerkelijk gemeten waarden uit de uitgevoerde tests. De resultaten zijn samengevat in tabellen en grafieken, zodat verschillen tussen de drie architecturen direct zichtbaar worden en gekoppeld kunnen worden aan de hoofdvraag.

5. Resultaten

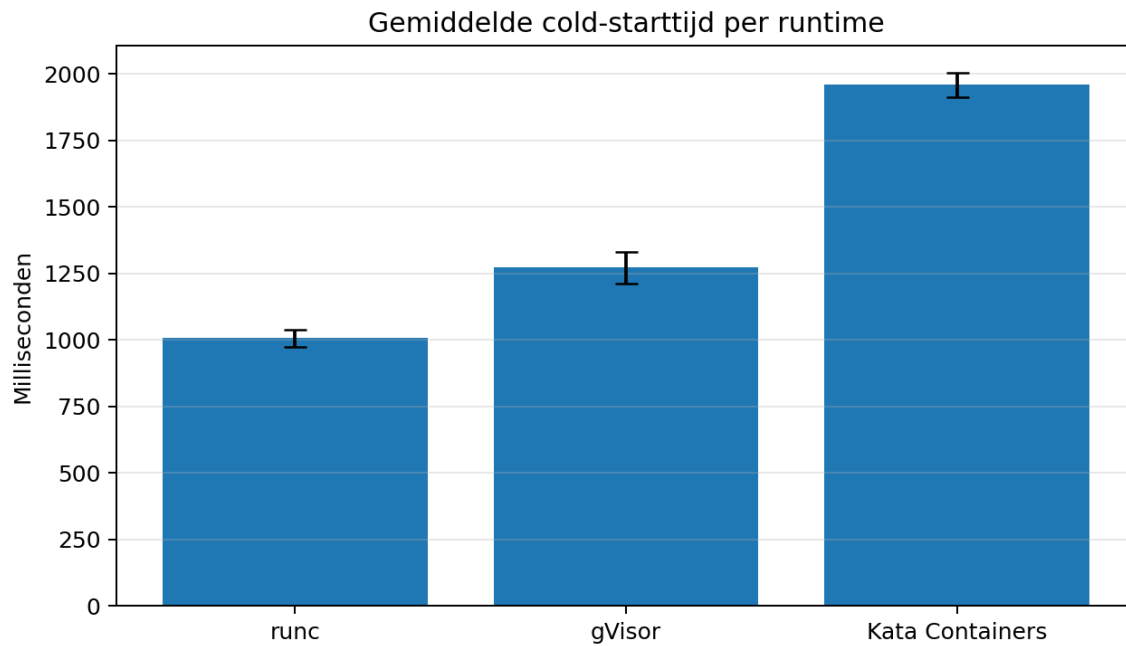
Dit hoofdstuk presenteert de uitkomsten van het eigen praktijkonderzoek. De meetbestanden bestaan uit drie onderdelen: cold-startmetingen in startup.csv, idle metingen in idle_stats.csv en loadtestruns met ApacheBench in combinatie met snapshots uit load_stats.csv. De hostinformatie is vastgelegd in system.txt. De tests zijn uitgevoerd op Debian 13 met Docker Engine 28.4.0, runsc release-20260323.0 en kata-runtime 3.28.0 op een host met 12 CPU-threads en 31.2 GiB RAM.

5.1 Cold-starttijd

Tabel 1 toont de cold-startresultaten per runtime op basis van twintig runs per architectuur. De cold-starttijd is gemeten als de tijd tussen het startcommando en de eerste succesvolle respons op het health-endpoint.

Architectuur	n	Gemiddelde starttijd (ms)	Standaarddeviatie (ms)	Min-max (ms)
runc	20	1006.4	30.7	963-1072
gVisor	20	1272.4	59.0	1155-1374
Kata Containers	20	1959.7	46.2	1856-2037

Figuur 1 visualiseert deze gemiddelden met foutbalken op basis van de standaarddeviatie.



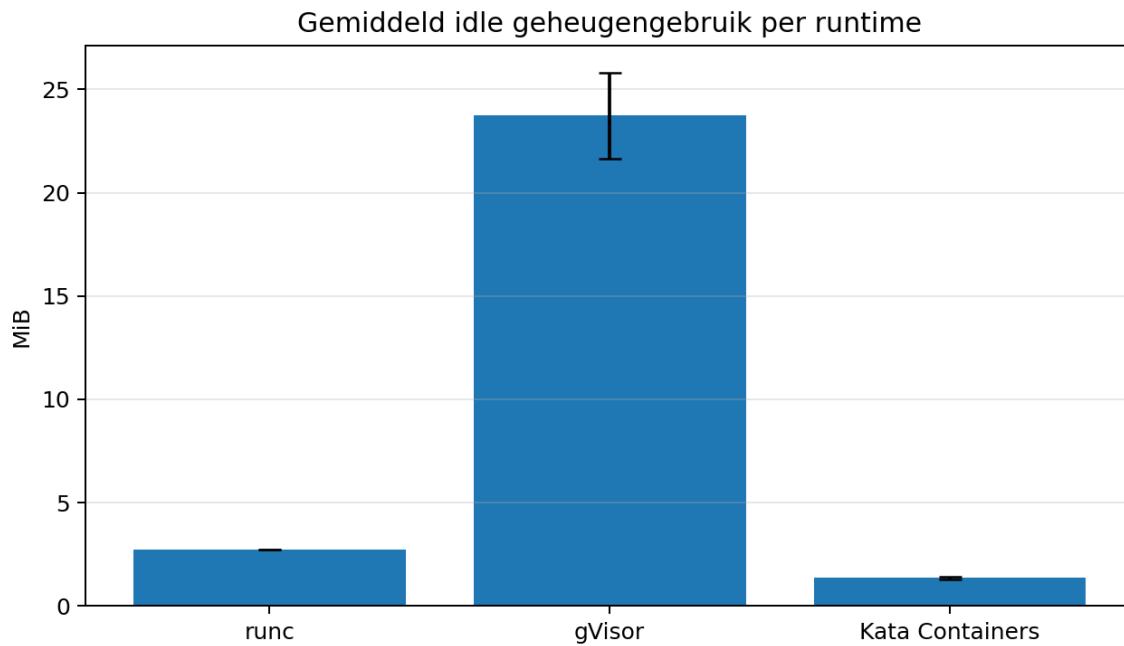
Figuur 1. Gemiddelde cold-starttijd per runtime.

runc startte gemiddeld het snelst (1006.4 ms). gVisor volgde op afstand (1272.4 ms), terwijl Kata Containers duidelijk de hoogste opstarttijd had (1959.7 ms). Daarmee bevestigen de cold-startmetingen de verwachting uit het literatuuronderzoek dat extra isolatielagen vooral merkbaar zijn bij het starten van workloads.

5.2 Idle resourcegebruik

Tabel 2 toont het idle geheugengebruik en het idle CPU-gebruik op basis van tien metingen per architectuur. Deze waarden zijn afkomstig uit `docker stats --no-stream` en moeten daarom gelezen worden als containergeattribueerde metingen, niet als volledige hostkosten.

Architectuur	n	Gem. geheugen (MiB)	Stdev geheugen	Gem. CPU (%)	Stdev CPU
runc	10	2.72	0.00	0.00	0.00
gVisor	10	23.73	2.09	6.32	0.32
Kata Containers	10	1.35	0.07	0.00	0.00



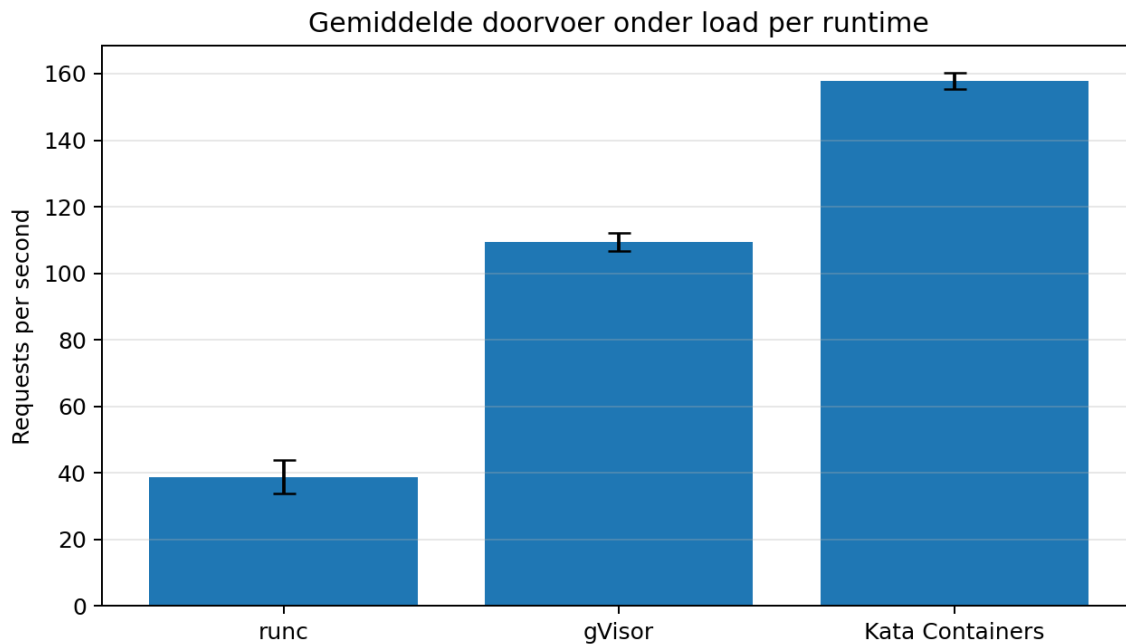
Figuur 2. Gemiddeld idle geheugengebruik per runtime.

Op basis van docker stats laat gVisor het hoogste idle geheugengebruik zien (23.73 MiB). runc blijft daar ruim onder (2.72 MiB). Kata Containers laat in deze meting zelfs de laagste containergeattribueerde geheugenwaarde zien (1.35 MiB). Juist dit resultaat vraagt om voorzichtigheid, omdat docker stats bij microVM-gebaseerde runtimes waarschijnlijk niet alle hostkosten volledig zichtbaar maakt.

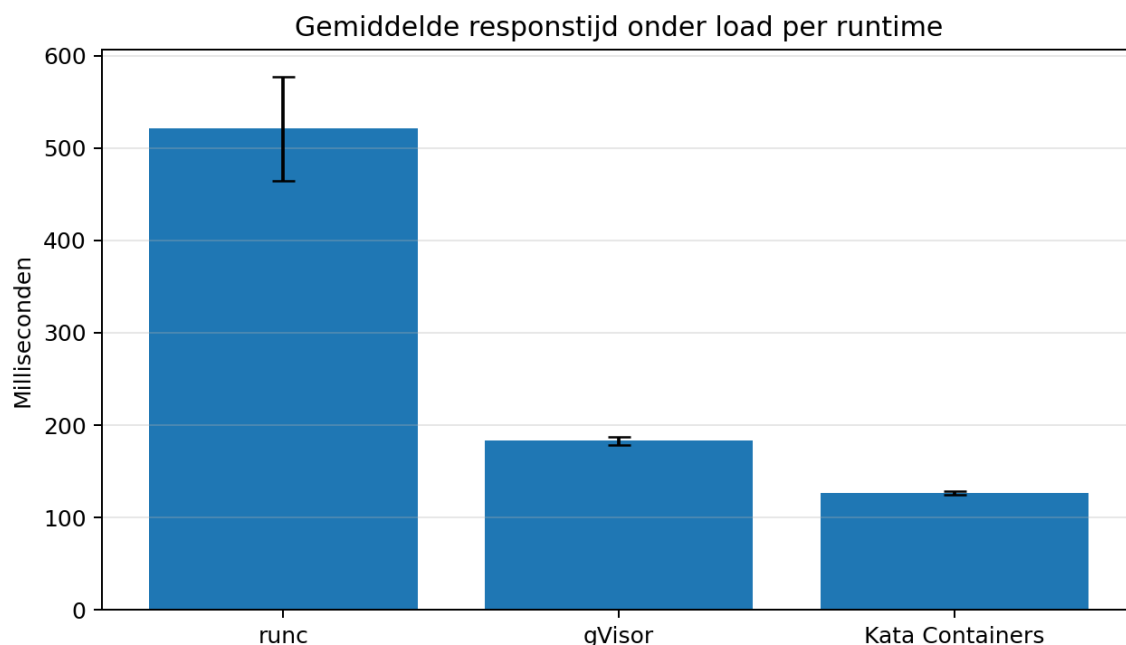
5.3 Prestaties onder belasting

Voor de loadtest zijn per runtime vijf ApacheBench-runs uitgevoerd op het /work-endpoint met 5000 requests en een concurrency level van 20. Tabel 3 toont de gemiddelde doorvoer, gemiddelde responstijd, gemiddelde P95-latency en het aantal mislukte requests.

Architectuur	n	Gem. requests/sec	Stdev requests/sec	Gem. responstijd (ms)	Gem. P95 (ms)	Failed requests totaal
runc	5	38.83	4.93	520.88	938.2	0
gVisor	5	109.32	2.67	183.03	391.6	0
Kata Containers	5	157.80	2.55	126.77	222.8	0



Figuur 3. Gemiddelde doorvoer onder load per runtime.



Figuur 4. Gemiddelde responstijd onder load per runtime.

Onder load behaalde Kata Containers de hoogste gemiddelde doorvoer (157.80 req/s) en de laagste gemiddelde responstijd (126.77 ms). gVisor eindigde in het midden met 109.32 req/s en 183.03 ms. runc bleef in deze proefopstelling duidelijk achter met 38.83 req/s en 520.88 ms. Geen van de vijftien loadtestruns bevatte mislukte requests.

De aanvullende snapshots uit load_stats.csv laten zien dat gVisor tijdens de loadmeting ook het hoogste containergeattribueerde geheugenverbruik had (gemiddeld 96.32 MiB). Voor runc en Kata Containers lagen deze snapshotwaarden veel lager, respectievelijk 13.03 MiB en 6.76 MiB. Ook hier geldt dat deze waarden voorzichtig geïnterpreteerd

moeten worden, omdat het om momentopnames gaat en niet om volledige hostmetingen.

6. Discussie

De resultaten laten zien dat de keuze voor een hostingarchitectuur niet met één enkel prestatiecijfer kan worden beantwoord. De uitkomsten verschillen namelijk per type metric. runc is het snelst bij cold starts en sluit daarmee aan op de verwachting uit het literatuuronderzoek dat standaardcontainers met gedeelde hostkernel de laagste opstartoverhead hebben. Tegelijkertijd laat dezelfde proefopstelling onder belasting juist een veel lagere doorvoer voor runc zien dan voor gVisor en Kata Containers.

Die uitkomst wijkt af van de eenvoudige verwachting dat meer isolatie automatisch slechtere prestaties betekent. Voor deze proof of concept is daarvoor geen sluitende technische verklaring beschikbaar, maar er zijn wel plausibele beperkingen. De benchmark gebruikte een kleine eigen testapplicatie met een CPU-belastend endpoint. Daardoor kan de concrete implementatie van de workload, de manier waarop de runtime met scheduling omgaat en de manier waarop het endpoint in de container draait relatief veel invloed hebben gehad op de loadtest. Het resultaat moet daarom gelezen worden als prestatiegedrag van deze specifieke opzet, niet als universeel oordeel over alle mogelijke applicaties.

Een tweede beperking zit in de resourcemetingen. Zowel `idle_stats.csv` als `load_stats.csv` zijn gebaseerd op `docker stats --no-stream`. Voor standaardcontainers geeft dat een bruikbare indicatie van containergebruik, maar voor microVM-gebaseerde runtimes zoals Kata Containers is het aannemelijk dat niet alle overhead van de onderliggende VM volledig aan de container wordt toegeschreven. Juist daarom is het opvallend lage idle geheugengebruik van Kata Containers waarschijnlijk geen volledige weergave van de werkelijke hostkosten. De geheugencijfers zijn dus nuttig als observatie, maar onvoldoende om op zichzelf een architectuurkeuze op te baseren.

Ook het CPU-gedeelte moet voorzichtig worden geïnterpreteerd. In de snapshots laten runc en Kata Containers tijdens idle en load steeds 0.00% CPU zien, terwijl gVisor wel een meetbare waarde laat zien. Dat suggereert dat de gekozen meetmethode niet gevoelig genoeg was om CPU-belasting op dezelfde manier voor alle drie de runtimes vast te leggen. Voor de beantwoording van de hoofdvraag wegen daarom vooral de cold-startmetingen en de ApacheBench-resultaten het zwaarst.

Voor Studententuin is vooral relevant dat het platform niet alleen snelle responstijden nodig heeft, maar ook beheersbaarheid, voorspelbaar resourcegebruik en een passende balans tussen efficiëntie en isolatie. Vanuit dat perspectief is runc aantrekkelijk voor scenario's waarin applicaties vaak gestart en gestopt worden en lage operationele complexiteit belangrijk is. Kata Containers laat juist zien dat sterkere isolatie niet per se leidt tot slechtere loadprestaties in deze proof of concept, maar daar staat een duidelijk

hogere cold-starttijd tegenover. gVisor eindigt in deze metingen vaak als middenpositie, maar kwam in dit experiment niet als duidelijke winnaar naar voren.

7. Conclusie en aanbevelingen

Op basis van de combinatie van literatuur en metingen kan niet worden gesteld dat één architectuur op alle punten superieur is. De metingen tonen een duidelijke trade-off. runc heeft de snelste cold starts en blijft daarmee het meest passend voor een eerste versie van Studententuin wanneer snelle opstart, eenvoud en een standaard containerworkflow centraal staan. Kata Containers behaalde echter de hoogste doorvoer en laagste responstijd onder de uitgevoerde loadtest, waardoor deze runtime interessant blijft voor scenario's waarin workloads langer actief blijven en sterkere isolatie gewenst is.

Voor een eerste MVP van Studententuin is runc daarom nog steeds de meest logische startkeuze. Die conclusie is niet gebaseerd op de loadtest alleen, maar op de bredere context van Studententuin als kleinschalig, betaalbaar en beheersbaar hostingplatform. Snelle starts en eenvoud wegen daarin zwaar. Tegelijkertijd laten de resultaten zien dat Kata Containers serieuzer onderzocht moet worden dan vooraf op basis van de literatuur misschien verwacht werd.

gVisor lijkt in deze meetreeks het minst overtuigende compromis. De runtime is trager dan runc bij cold starts en bleef onder load achter op Kata Containers, terwijl de gemeten containergeattribueerde geheugen- en CPU-waarden juist het hoogst waren. Dat betekent niet dat gVisor ongeschikt is, maar wel dat deze proof of concept geen sterke reden geeft om gVisor voor Studententuin boven de andere twee varianten te verkiezen.

De belangrijkste aanbeveling voor vervolgonderzoek is daarom om de metingen uit te breiden met hostniveau-observaties, bijvoorbeeld via systemd-cgtop, cAdvisor of aanvullende VM-metingen, zodat de werkelijke geheugen- en CPU-kosten van Kata Containers beter zichtbaar worden. Daarnaast is het aan te raden om meerdere typen workloads te testen, zoals een I/O-zware webapplicatie, een database-gekoppelde applicatie en een applicatie met lange uptime. Daarmee kan beter worden vastgesteld of het hier gemeten gedrag ook standhoudt buiten deze ene benchmark.

Een tweede aanbeveling is om de operationele aspecten explicieter mee te nemen in een vervolgstudie. Voor Studententuin zijn niet alleen pure performancecijfers belangrijk, maar ook compatibiliteit, foutafhandeling, debuggemak, logging en beheercomplexiteit. Juist op die punten kan een architectuur die technisch net iets minder snel is in de praktijk toch de beste keuze blijken.

Bronnenlijst

Docker. (2026). What is a container? Docker Docs.

Docker. (2026). Alternative container runtimes. Docker Docs.

gVisor. (2026). What is gVisor? gVisor Documentation.

gVisor. (2026). Docker quick start. gVisor Documentation.

gVisor. (2026). Performance guide. gVisor Documentation.

Kata Containers. (2026). About Kata Containers. katacontainers.io.

Agache, A., Brooker, M., Iordache, A., Liguori, A., Neugebauer, R., Piwonka, P., & Popa, D. (2020). Firecracker: Lightweight virtualization for serverless applications. USENIX NSDI 2020.